Ansible Playbooks - 2 Advanced Structure & Components

Mastering the essential building blocks of Ansible playbooks for robust infrastructure automation and configuration management.

Variables: The Foundation of Dynamic Playbooks

Variables in Ansible playbooks store values that can be reused throughout your automation, making playbooks more dynamic and flexible by allowing values to change based on different conditions or inputs.

Inventory Files

Define host-specific variables directly in your inventory configuration

Playbooks & Roles

Embed variables within playbook structure or role definitions

External Sources

Load from YAML, JSON files, or command line with --extra-vars

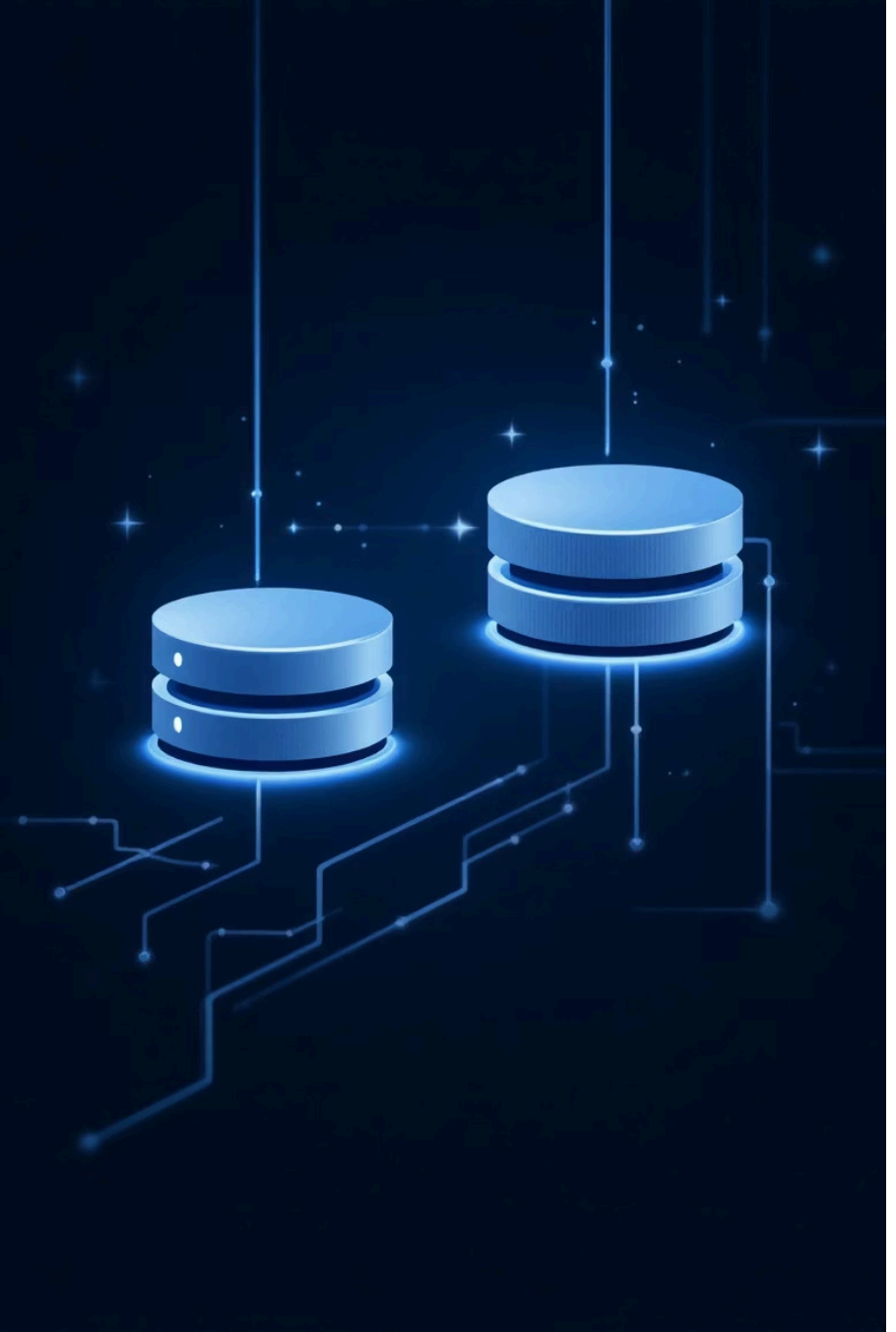
```
ansible-playbook -i inventory playbook.yaml --extra-vars
"password=Gavina"
```

Roles: Reusable Automation Components

Code Reusability at Scale

Roles enable you to write automation code once and reuse it across multiple playbooks, promoting consistency and reducing maintenance overhead.

Critical execution order: Roles execute before any other tasks in a play, ensuring proper dependency management and configuration sequencing.



Handlers: Event-Driven Task Execution

Handlers are special tasks triggered by other tasks when changes occur, typically used for actions like restarting services after configuration modifications.

01

Task Execution

Primary task runs and reports a change to the system

02

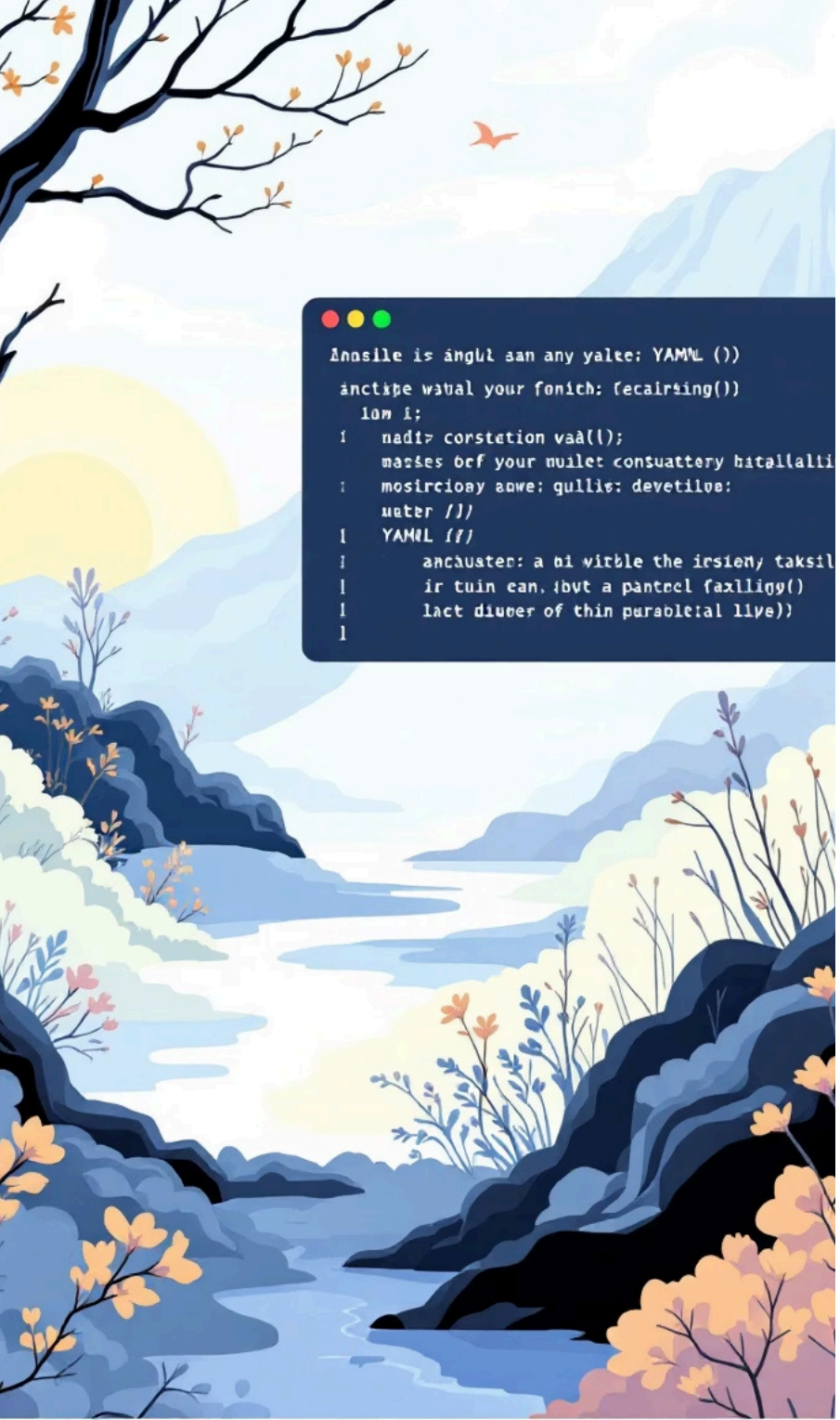
Handler Notification

Task uses notify keyword to trigger specific handlers

03

End-of-Play Execution

Handlers execute at play completion, after all tasks finish



Tasks: Core Playbook Components

Tasks represent individual actions Ansible performs on target hosts. Each task includes a fully qualified collection name (FQCN) with module parameters.

FQCN Format: Separate collection name from module with a dot, e.g., `ansible.builtin.yum`

Module Selection

Choose appropriate modules from collections for specific automation tasks

Parameter Configuration

Define module-specific parameters to control task behaviour and outcomes

Includes vs Imports: Dynamic Task Management

Include Tasks

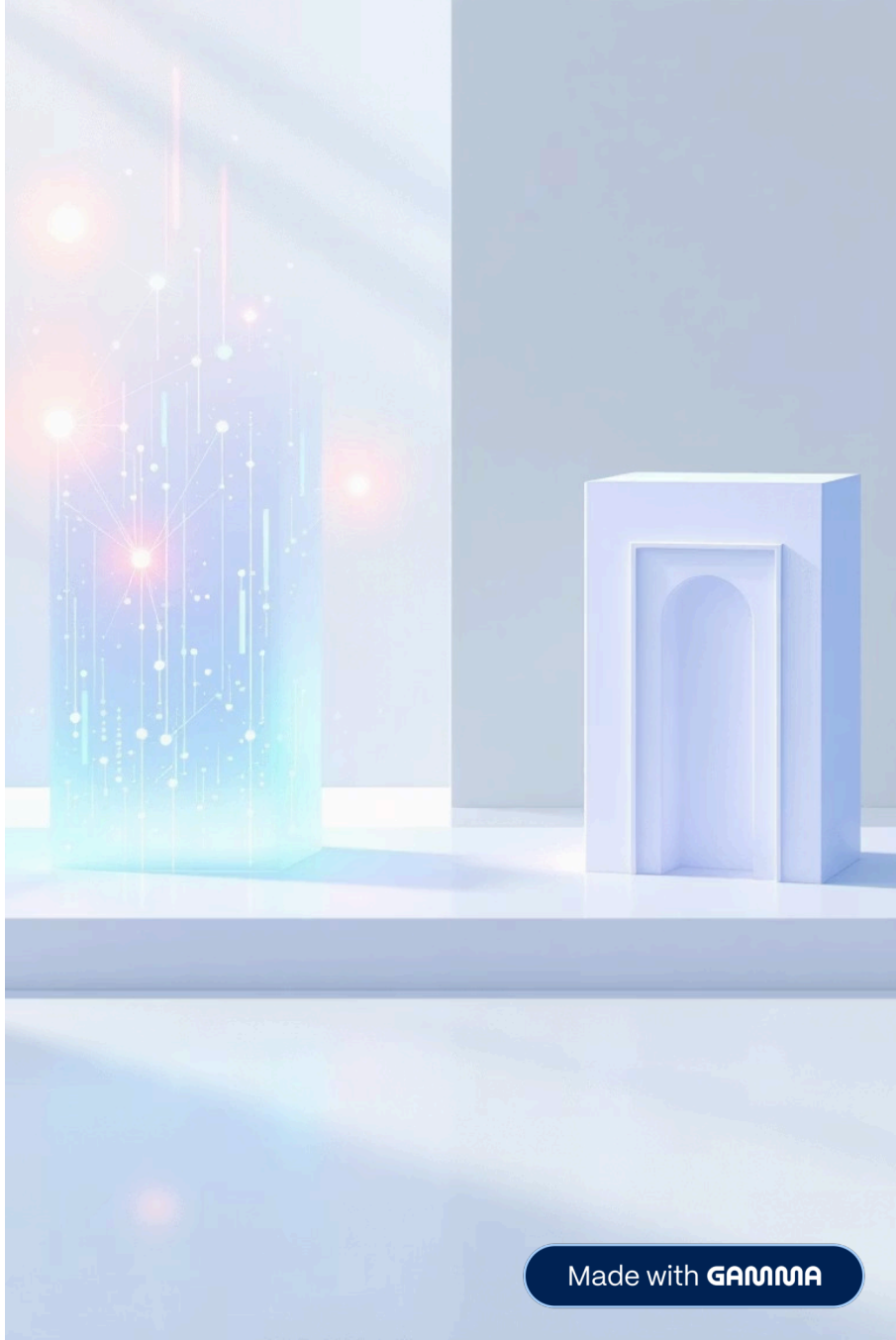
Runtime execution - Dynamic and flexible, processed during playbook execution

```
tasks:
  - ansible.builtin.include_tasks: db.yml
```

Import Tasks

Parse time - Static inclusion, processed before execution begins

```
tasks:
 - ansible.builtin.import_tasks: web.yml
```



Advanced Playbook Features

Blocks

Group related tasks together with common attributes like error handling or conditional execution. Support nested grouping for complex task organisation.

Tags

Label tasks, plays, or roles for selective execution. Use `--tags` to run specific parts or `--skip-tags` to exclude sections.

Register

Capture task output in variables for use in subsequent tasks. Essential for conditional logic and data flow between tasks.

Collections: Organised Content Distribution

Collections package and distribute Ansible content including modules, plugins, roles, and playbooks in a structured format.

1

Installation

Install from Ansible Galaxy using `ansible-galaxy` command-line tool

2

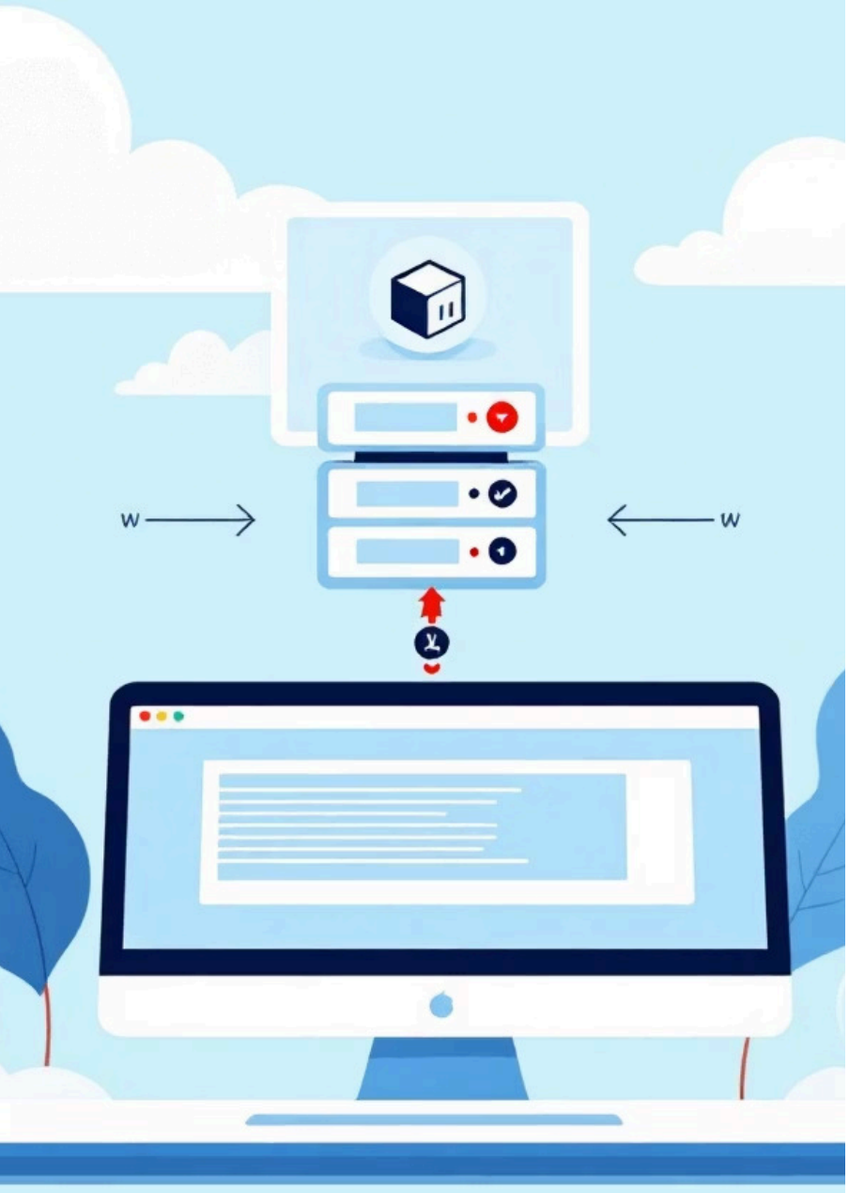
Organisation

Structured packaging makes content easier to manage and reuse across projects

3

Usage

Reference modules using fully qualified collection names (FQCN) in playbooks



Your First Playbook: Web Server Configuration

A practical example demonstrating core Ansible concepts through automated web server setup and configuration management.

Install Nginx

Deploy web server package using appropriate package manager

Configure Service

Apply nginx configuration settings and copy HTML content to web directory

Apply Changes

Restart nginx service and configure firewall rules for web access

Validate Deployment

Test web page accessibility and verify successful configuration

Execute Your Playbook

```
ansible-playbook -i inventory -K playbook.yaml
```

Command Breakdown

- `-i inventory` - Specify inventory file location
- `-K` - Prompt for sudo password authentication
- `playbook.yaml` - Target playbook file to execute

Next steps: Monitor execution output, verify configuration changes, and iterate on your automation as requirements evolve.